NATIONAL AND KAPODISTRIAN UNIVERSITY OF ATHENS
DEPARTMENT OF INFORMATICS AND TELECOMMUNICATIONS

Deep Learning for NLP

Student name: *Georgios Xydias*
ID: 7115152400007

Course: *Artificial Intelligence II (YS19 M226 M262 M325 M405)*
Semester: *Spring Semester 2025-2026*

D3-Agentic Prompting for Response Clarity Classification

Contents

1. Abstract

This report presents our approach to response clarity classification in political question-answer pairs, as part of SemEval 2026 Task 6 [8]. The dataset, introduced by [7], consists of political interviews with U.S. presidents, where each answer is labeled as *Clear Reply*, *Ambivalent*, or *Clear Non-Reply*. We approach the task through a multi-agent prompting framework (D3-Agentic Prompting) applied to an instruction-tuned large language model, rather than a single prompt or gradient-based fine-tuning. The implementation uses PyTorch [4] and the Hugging Face Transformers library [9]. We use Qwen3.5-0.8B from the Qwen3 family [10] as the underlying model for all four agents. The framework decomposes the classification task into four specialized agents: a Question Intent agent that identifies what the question asks for, an Answer Content agent that extracts what the answer says, a Gap and Evasion agent that compares the two and detects missing or evasive content, and a Decision agent that assigns the final label. We evaluate the agentic pipeline systematically, analyze failure modes per agent, and compare the results with our previous work, which evaluated classical machine learning baselines (TF-IDF and GloVe with Logistic Regression), fine-tuned encoder transformers (BERT, DistilBERT, DeBERTa), and single-prompt LLM approaches. Our approach is informed by the agentic AI taxonomy of [5], the prompting reference [6], and the relevant chapters of [2]. All implementation details, additional figures, and intermediate results are available in the accompanying Jupyter notebook.

2. Data

2.1. Dataset Description

We use the dataset introduced by [7] for SemEval 2026 Task 6 [8], which consists of political interviews with U.S. presidents. Each record contains the original journalist turn (`interview_question`), the politician's response (`interview_answer`), annotation information, metadata (e.g., title, date, president, URL), and auxiliary fields such as GPT-3.5 generated summaries. Because a single journalist turn may contain multiple questions, the dataset provides a segmented `question` field containing exactly one question. The core supervised instances in this work are therefore (`question`, `interview_answer`) pairs. The prediction target is `clarity_label`, a three-class label: *Clear Reply*, *Ambivalent*, and *Clear Non-Reply*. The dataset is available on Hugging Face under the identifier `ailsntua/QEvasion`, and provides two splits: 3,448 training samples and 308 test samples.

2.2. Exploratory Data Analysis

The full exploratory data analysis of this dataset is described in our previous work; here we summarize only the aspects that directly affect the prompting pipeline.

The class distribution is highly imbalanced: *Ambivalent* accounts for 59% of the training samples, *Clear Reply* for 31%, and *Clear Non-Reply* for only 10% (Figure 1). The class distribution is consistent between the training and test splits. This imbalance is relevant when selecting demonstrations or calibration examples, where balancing classes matters.

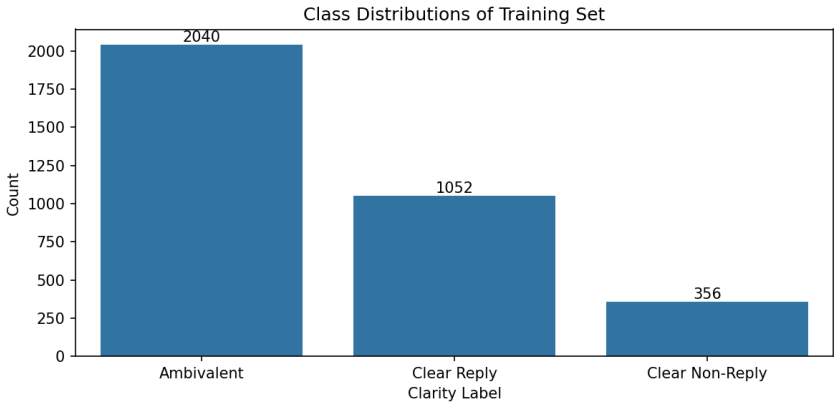

Figure 1: Class distribution in the training set.

Text length varies across classes: *Ambivalent* answers tend to be the longest (mean 331 words), followed by *Clear Reply* (272 words), and *Clear Non-Reply* (137 words). Context length is not a limitation in this work: even the longest combined input fits comfortably within the model's context window.

2.3. Text Preprocessing

As pretrained large language models operate on raw text and rely on their own pretrained tokenizers, only one text preprocessing step was applied: Deduplication. We identified and removed 58 duplicate rows (1.68%) from the training set, reducing it from 3,448 to 3,390 samples. This prevents the same example from appearing in both the few-shot demonstration pool and the held-out validation set, which would affect the validation estimates.

Unlike our earlier work with fine-tuned encoder transformers, which used a fixed input format applied uniformly, no global input construction is required here. Tokenization is handled internally by the model's chat template.

2.4. Data Partitioning

We use the same 85/15 random split as in our previous works on this dataset, resulting in a 2,881-sample few-shot demonstration pool and a 509-sample held-out validation set. Because the split is regenerated with stratification, the validation set is close to but not identical to the one used in our previous work, allowing a meaningful comparison across assignments. The test set (308 samples) is kept untouched until final prediction. A fixed random seed (`SEED=42`) is used throughout all experiments to ensure reproducibility.

3. Model

Prompting is a way of using pretrained large language models in which the task is specified entirely through the input text, with no updates to the model's parameters [2]. The model receives a prompt that frames the problem and an input instance, and returns a textual response that is parsed into a prediction. This contrasts with fine-tuning (used in our earlier work with encoder transformers), where task-specific labeled data

updates the model's weights through gradient descent. The same pretrained model can therefore handle many tasks by changing only the prompt, which is why we access it in its instruction-tuned variant (specifically trained to follow natural-language task instructions).

We use Qwen3.5-0.8B [10], the same instruction-tuned model used in our previous work on this dataset. The model is a decoder-only transformer with approximately 0.75 billion parameters distributed across 24 transformer blocks, each with 8 attention heads operating on 1024-dimensional hidden states. The tokenizer has a vocabulary of 248K subword tokens. The model supports a context window of 262,144 tokens, which is far more than this task requires: even the largest single agent call stays well under a few thousand tokens. The model is loaded in `bfloat16` precision (16 bits per weight) on a single GPU from the Hugging Face Transformers library [9].

4. D3-Agentic Prompting

D3-Agentic Prompting is a multi-agent framework in which the response clarity classification task is decomposed into a chain of four specialized agents. Instead of asking a single LLM prompt to interpret the question, analyze the answer, compare them, and assign a label all at once, we delegate each of these subtasks to a separate agent with a focused system prompt. Each agent produces an intermediate output that is appended to a shared context and passed forward, so downstream agents reason over an accumulating chain of analyses. The full pipeline is shown in Figure 2.

The four agents are: a *Question Intent* agent that identifies what information a clear answer would need to address, an *Answer Content* agent that extracts what the answer actually says, a *Gap and Evasion* agent that compares the two and characterizes any discrepancies, and a *Decision* agent that issues the final clarity label. All four agents share the same base model (Qwen3.5-0.8B) and the same chat-template interface, but each receives a different system prompt that constrains its behavior. A deliberate design choice in our implementation is that every agent, including the downstream ones, has access to the original (question, answer) pair, not only the outputs of preceding agents. This is intended to limit the propagation of upstream errors: if an early agent misreads the input, the later agents may still recover by consulting the source text directly.

4.1. Question Intent Agent

The Question Intent Agent is the first stage of the pipeline. It receives only the journalist's question (the `question` field from the dataset) as user input. The politician's answer is excluded so that the analysis is not influenced by how the politician chose to respond.

The agent produces a structured JSON object with five fields: `question_type` (a categorical tag such as `yes_no`, `stance`, or `why`), `question_intent` (one sentence describing what the question asks), `must_address` (a list of concrete requirements a clear answer must satisfy), `clear_answer_criteria` (one sentence describing what a clear answer must contain), and `minimal_sufficient_answer` (the minimum information needed for a Clear Reply). Together, these fields establish the criteria against which the politician's answer will later be assessed by the Gap and Evasion Agent (Section 4.3).

The system prompt (Appendix A.1) instructs the agent to think in terms of answer requirements: whether the question asks for a yes/no, an explanation, a stance, a spe-

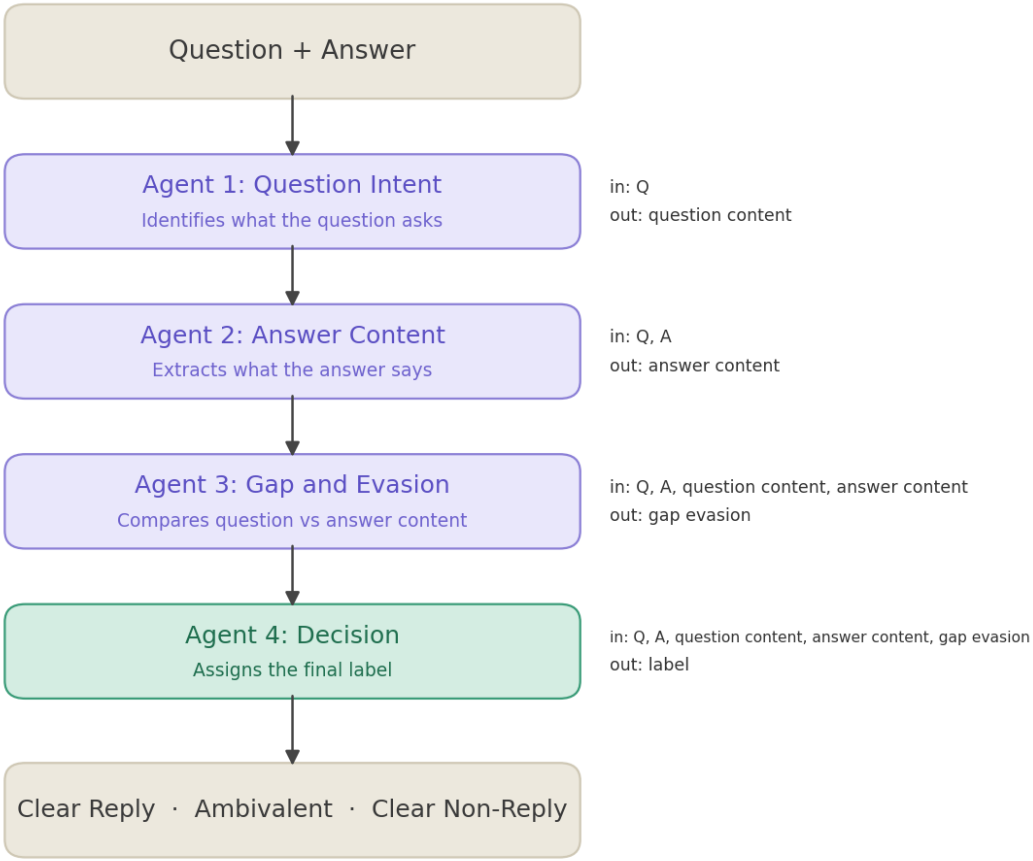

Figure 2: D3-Agentic Prompting pipeline. The question and answer pass through four agents: Question Intent, Answer Content, Gap and Evasion, and Decision. Each agent's intermediate output accumulates and feeds the next, ending in a clarity label. Agents 1–3 produce analyses while Agent 4 produces the final classification.

cific action, and so on. It also includes the constraint "Return exactly one JSON object and nothing else", which keeps the output machine-parseable and prevents the model from skipping ahead to classification. Without this constraint, small instruction-tuned models tend to produce a label or a guessed answer instead of analyzing the question, which would mislead the later agents. Generation is greedy (`do_sample=False`) and capped at 256 new tokens, which is sufficient for the short JSON analyses and tight enough to prevent verbose elaboration.

4.2. Answer Content Agent

The Answer Content Agent receives the question and the politician's answer. Its task is to extract what the answer actually says that is relevant to the target question, without deciding the final label. The question is included so the agent can judge relevance, but the focus is on describing the answer itself.

The agent produces a structured JSON object with seven fields: `relevant_answer_summary` (one sentence summarizing only content relevant to the target question), `direct_answer_present` (a boolean indicating whether the answer literally resolves the question), `direct_answer_text` (the exact short phrase if present), `concrete_information` and `vague_or_general_information`

(lists separating hard facts from softer claims), `off_target_content` (content that addresses a different question or shifts topic), and `refusal_or_non_answer_signals` (phrases such as "I will not discuss" or "we'll see"). This separation between concrete, vague, off-target, and refusal content gives the downstream Gap and Evasion Agent (Section 4.3) structured evidence rather than a free-text summary to work with.

The system prompt (Appendix A.2) includes several negative constraints: do not decide the final label, do not reward political rhetoric unless it answers the question, do not infer facts not present in the answer, and do not import topics from the question into the extraction. The last point is particularly important: if the answer does not mention a topic the question raises, the agent must leave it out entirely, because its absence is exactly what the Gap and Evasion Agent must detect. Without these constraints, small instruction-tuned models tend to "fill in the blanks" by importing question topics into their extraction, which masks evasion. Generation is greedy and capped at 512 new tokens, higher than Agent 1 because politician answers can be long and the JSON output has more fields.

4.3. Gap and Evasion Agent

The Gap and Evasion Agent is the comparison stage of the pipeline. It receives four inputs: the target question, the politician's answer, the Question Intent Agent output, and the Answer Content Agent output. Its task is to compare what the target question requires with what the answer provides, and to characterize any mismatch.

The agent produces a structured JSON object with seven fields. The first two are ordinal scales: `coverage` (`complete`, `mostly_complete`, `partial`, `minimal`, or `none`) and `gap_level` (`none`, `minor`, `partial`, or `major`). The third field, `best_evasion_category`, assigns one of ten fine-grained evasion labels drawn from the political communication literature: `explicit_answer`, `implicit_answer`, `partial_half_answer`, `general_answer`, `dodging`, `deflection`, `declining_to_answer`, `claims_ignorance`, `clarification_only`, and `irrelevant`. The remaining fields are lists of evidence: `missing_requirements`, `evidence_for_answering`, `evidence_for_evasion_or_gap`, and a one-sentence `gap_explanation`. Together, these give the Decision Agent (Section 4.4) both a structured signal (the coverage and gap scales) and grounded evidence (the lists) to base its label on.

The system prompt (Appendix A.3) makes an important three-way distinction that guides the agent's judgment: missing a small detail while mostly answering indicates a minor gap, providing some relevant information that is vague, incomplete, or indirect indicates a partial gap, and mostly answering another topic, refusing, or avoiding the question indicates a major gap. This graduated scale is intended to help the Decision Agent distinguish between all three classes, especially cases where the answer contains some relevant information but falls short of fully resolving the question. As with the previous agents, the prompt prohibits assigning the final label and requires exactly one JSON object. Generation is greedy and capped at 512 new tokens.

4.4. Decision Agent

The Decision Agent is the final stage of the pipeline. It receives five inputs: the target question, the politician's answer, and the three structured JSON outputs from Agents 1, 2, and 3. Using the upstream analyses as its primary signal and the raw

question–answer pair for grounding, it assigns exactly one label: *Clear Reply*, *Ambivalent*, or *Clear Non-Reply*.

The agent produces a JSON object with four fields: `label` (one of the three valid classes), `confidence` (a float between 0 and 1), `main_reason` (one concise sentence justifying the choice), and `used_rule` (a short description of the rule that determined the label). Although only `label` is used for evaluation, the other three fields serve an interpretability purpose: they let us inspect *why* the agent chose a particular class, which is important for error analysis.

The system prompt (Appendix A.4) contains three components beyond the label definitions. First, it provides explicit decision rules that map the Gap and Evasion Agent's `coverage` field to labels: `complete` or `mostly_complete` coverage maps to Clear Reply, `partial` or `minimal` coverage with relevant information maps to Ambivalent, and `none` or mainly refusal/deflection maps to Clear Non-Reply. These rules are intended to give the 0.8B model a deterministic backbone for the decision rather than relying on its own judgment of the raw text. Second, it includes tie-breaking preferences for borderline cases: prefer Ambivalent over Clear Non-Reply when the answer has relevant but insufficient information, and prefer Ambivalent over Clear Reply when the answer is related but lacks the requested specifics. Third, it provides five few-shot calibration examples embedded directly in the prompt, each showing a target question, an answer-content summary, a gap-analysis summary, and the correct label. These examples anchor the model's calibration across the three classes without requiring separate demonstration sampling.

Generation is greedy and capped at 384 new tokens, which is sufficient for the JSON output. The `label` field is extracted from the parsed JSON and normalized by a shared label parser described in Section 4.5.

4.5. Pipeline Execution and Output Parsing

The four agents are chained into a single function that processes one (question, answer) pair at a time. For each example, the pipeline runs Agent 1 through Agent 4 sequentially: each agent's JSON output is serialized and included in the user message of the next agent. There is no batching across examples as each agent call is a single `model.generate` invocation under `torch.inference_mode()`, using greedy decoding (`do_sample=False`) and the model's chat template (`apply_chat_template` with `add_generation_prompt=True`). The per-agent token budgets are 256, 512, 512, and 384 for Agents 1 through 4 respectively. Partial results are checkpointed to disk every 25 examples so that interrupted runs can be resumed.

Output parsing is shared across all four agents and operates in two stages. First, a JSON extractor strips any ‘‘‘`json` fences, attempts to parse the full response as JSON, and falls back to extracting the first `{...}` block if the response contains surrounding text. If no valid JSON object is found, the extractor returns a dictionary with only the raw text, which downstream code treats as a parse failure. Second, for the Decision Agent specifically, a label normalizer maps the `label` field to one of the three valid classes. The normalizer removes any `<think>...</think>` reasoning blocks, lowercases the text, replaces underscores with spaces, and matches against keywords in priority order: “non” plus “reply” maps to Clear Non-Reply, “ambivalent” (or “ambiguous”, “partial”, “unclear”) maps to Ambivalent, and “clear reply” (or standalone “reply”) maps to Clear Reply. If no keyword matches, the normalizer returns `None`,

and the prediction is counted as a parse failure. For evaluation, parse failures are left as `None` and excluded from the per-class metrics. For the Kaggle submission only, they are filled with Ambivalent (the majority class).

5. Experiments and Results

5.1. Experimental Setup

All experiments were run on Google Colab Pro with a single NVIDIA A100 40 GB GPU, with models loaded in `bfloat16` precision. The full set of experiments consumed approximately 200 A100 compute units, which limited the number of configurations we could explore so the experiments reported below reflect the variants we prioritized within this budget. A fixed random seed (`SEED=42`) was set across Python, NumPy, PyTorch, and Hugging Face Transformers to ensure reproducibility of the data split, demonstration sampling, and greedy generation.

Because the D3-Agentic pipeline requires four sequential model calls per example (one per agent), inference is substantially slower than the single-call techniques (batched approach) evaluated in our previous work. To keep the experimental cycle practical within the available compute budget, all validation experiments were run on a stratified 100-row subset of the 509-sample validation set described in Section 2.4. The test set (308 samples) was kept untouched until the final evaluation of the winning configuration.

We report four metrics for each experiment: accuracy, macro-averaged F1 (which weights all three classes equally and is therefore sensitive to minority-class performance), weighted-averaged F1 (which weights by class support), and parse failure rate (the fraction of examples for which the output parser could not extract a valid label). Parse failures are left as invalid predictions during evaluation, meaning they count as errors for the true class. Per-class precision, recall, and F1 are reported separately in the Evaluation section (Section 6) for the winning configuration.

5.2. Experimental Process

We began by establishing the baseline: the full four-agent pipeline with the reference prompts described in Section 4, run on the validation subset. The system achieved a macro F1 of 0.455 and a weighted F1 of 0.569, with no parse failures. Inspecting the per-class predictions, Ambivalent was the best-recognized class (F1 = 0.65), followed by Clear Reply (F1 = 0.49), while Clear Non-Reply was poorly recovered (F1 = 0.22, with only 1 of 7 true instances correctly identified). Most errors involved confusion between Clear Reply and Ambivalent, suggesting that the Decision Agent struggles to draw the line between partial and sufficient answers.

Since the baseline errors concentrated on the Decision Agent's boundary between Clear Reply and Ambivalent, we first tried the approach that had helped most in our previous single-prompt work: adding few-shot demonstrations. We built 9 class-balanced demonstrations (3 per class) from the pool set, running each through Agents 1–3 so the demonstration payload matched what the Decision Agent sees at inference time. However, this hurt performance: macro F1 dropped from 0.455 to 0.384 and weighted F1 from 0.569 to 0.476. The additional demonstrations made the Decision Agent's input substantially longer, and the 0.8B model appeared to lose track of the

structured signals from the upstream agents among the extra context. This indicated that the issue was not a lack of examples for the Decision Agent, and prompted us to investigate which upstream agents were actually contributing most to the pipeline.

To understand which agents were contributing most, we ran two ablation experiments, keeping the Decision Agent (Agent 4) in both since it produces the final label. In the first ablation, we dropped the Gap and Evasion Agent (Agent 3), keeping Agents 1, 2, and 4. Macro F1 fell from 0.455 to 0.389, confirming that the gap-analysis step provides useful signal to the Decision Agent. In the second ablation, we dropped both the Question Intent Agent and the Gap and Evasion Agent (Agents 1 and 3), keeping only the Answer Content Agent (Agent 2) and the Decision Agent. Macro F1 reached 0.426, close to the full pipeline's 0.455. This was a surprising result: the simpler two-agent pipeline nearly matched the full four-agent system, suggesting that Agent 2's structured extraction of what the answer says already captures most of the signal the Decision Agent needs, and that Agent 3's comparison step, while helpful, is not as critical as expected. Reading the two ablations together, the Question Intent Agent (Agent 1) appears to contribute relatively little to the final decision, while the Gap and Evasion Agent (Agent 3) accounts for the largest share of the improvement over the simpler pipelines. These results pointed us toward Agent 3 as the component with the most room for improvement.

Before attempting to improve Agent 3's prompt, we tested a more extreme simplification: replacing Agents 1, 2, and 3 entirely with a single Merged Analysis Agent that reads the question and answer directly and produces the gap-and-evasion JSON in one step, which then feeds into the Decision Agent. This two-agent variant achieved an accuracy of 0.61 (the highest of any configuration) but a macro F1 of only 0.407, lower than the full pipeline's 0.455. The gap between the two metrics indicates that the merged agent favors the majority class (Ambivalent) at the expense of the minority classes. This confirmed that the three-agent decomposition before the Decision Agent does add value for balanced class recognition, even if the individual contribution of each agent is modest, and that improving Agent 3's prompt rather than removing it was the right direction.

Guided by the ablation results, we manually revised the prompts for the two agents that influenced the final score most: the Gap and Evasion Agent (Agent 3) and the Decision Agent (Agent 4). The revised prompts were stricter: Agent 3's v2 prompt replaced the five-level coverage scale with a simpler three-level one (`complete`, `partial`, `none`) and added explicit instructions to penalize off-topic answers. Agent 4's v2 prompt mapped these three coverage values directly to the three labels and listed common evasion patterns (postponing, refusing, claiming ignorance) that should always count as Clear Non-Reply. The full v2 prompts are given in Appendix A.5. We tested three combinations: revising Agent 3 alone (macro F1 = 0.325), Agent 4 alone (0.263), and both together (0.359). All three scored below the baseline (0.455). The revised prompts were too aggressive: they classified too many answers as Clear Non-Reply, hurting performance on the other two classes. Since manual prompt engineering in both directions (looser with few-shot examples, stricter with rewritten rules) had failed to improve the baseline, we turned to automatic prompt optimization.

Since manual prompt revision had not improved the baseline, we explored automatic prompt optimization using DSPy [3, 1]. DSPy requires describing each agent as a *Signature* (inputs, outputs, and a task docstring) wrapped in a *Module*, together with a scoring metric. An optimizer then searches for better prompts or demonstrations that

maximize the metric. We tested three optimizers: BootstrapFewShot, which runs the model on training examples and collects successful outputs as few-shot demonstrations, BootstrapFewShotWithRandomSearch (RandomSearch), which generates multiple candidate demonstration sets and selects the best, and MIPROv2, which additionally rewrites the prompt instructions alongside selecting demonstrations. Because our agents run on a local model rather than an API, we built a simple wrapper that routes DSPy's OpenAI-shaped calls to our `generate_chat` function. The scoring metric for Agents 1 and 3 was end-to-end correctness: the optimized agent is plugged into the full pipeline, and the metric returns 1.0 if the final label matches the gold label, 0.0 otherwise. For Agent 4, we used a class-weighted variant that returns 3.0 for a correct Clear Non-Reply and 1.0 for other correct predictions, to prevent the optimizer from ignoring the rare minority class. We created Signatures and Modules for Agents 1, 3, and 4 (Agent 2 was kept fixed since the ablations showed it already performs well) and tested each optimizer in multiple configurations, varying the number of bootstrapped demonstrations and the target agent. No DSPy-optimized variant exceeded the reference-prompt baseline. Increasing the number of demonstrations consistently hurt performance, consistent with the few-shot Decision Agent result earlier, where adding demonstrations to the input also hurt performance. The hand-crafted reference prompts, which include task-specific decision rules and calibration examples, were already well-suited for this model size, and automatic optimization could not compensate for the model's limited capacity to process demonstration-heavy inputs.

To test whether the D3-Agentic pipeline generalizes beyond Qwen3.5, we ran the full four-agent system with the same reference prompts on two additional instruction-tuned models of comparable size: Llama-3.2-1B-Instruct and Gemma-3-1B-it. Llama-3.2-1B reached a macro F1 of 0.163, far below the Qwen3.5-0.8B baseline of 0.455. Gemma-3-1B performed better (macro F1 = 0.293) but still fell short of Qwen3.5-0.8B, and showed a 12% parse failure rate, indicating that it struggled to produce valid JSON consistently. These results suggest that the reference prompts, which were designed and tuned on Qwen3.5-0.8B, do not transfer well to other model families without prompt adaptation. A fairer comparison would require re-tuning the prompts for each model, which was beyond our compute budget.

The experiments above follow a set of practical constraints. We treated the reference prompts as the fixed starting point rather than designing alternatives from scratch. We kept the four-agent architecture throughout, even though the two-agent ablation showed competitive results. We used models of comparable size (0.8B–1B) across all families, without substituting larger models for individual agents. These choices limited the space of configurations we explored but ensured that each experiment isolated a single variable against a consistent baseline. Further extensions, including the use of larger models for critical agents and mixed-size pipelines, are discussed in Section 8.

5.3. Results Summary

Table 1 collects all D3-Agentic experiments on the 100-row validation subset. The full pipeline with reference prompts on Qwen3.5-0.8B achieved the highest macro F1 (0.455) and weighted F1 (0.569) among all configurations tested. This configuration was selected as the winning system and evaluated on the held-out 308-row test set. No variant (few-shot demonstrations, manual prompt revision, DSPy optimization, agent

ablation, merged architecture, or alternative model family) exceeded this baseline. The per-class evaluation of this winning configuration and its performance on the held-out test set are presented in Section 6.2.

Table 1: All D3-Agentic experiments on the 100-row validation subset. The best score in each metric column is in bold. All experiments use Qwen3.5-0.8B unless noted otherwise.

Configuration	Description	Acc	Macro F1	Weighted F1	Parse fail
Pipeline variants					
D3 (full)	Reference prompts	0.57	**0.455**	**0.569**	0%
D3 (full)	Few-shot decision (9 demos)	0.47	0.384	0.476	0%
D3 (drop Gap)	Ablation: drop Agent 3	0.48	0.389	0.456	0%
D3 (drop Intent & Gap)	Ablation: drop Agents 1, 3	0.52	0.426	0.490	0%
D3 (merged)	Single merged agent + Decision	**0.61**	0.407	0.563	0%
Manual prompt revision					
D3 (full)	Agent 3 v2	0.40	0.325	0.307	0%
D3 (full)	Agent 4 v2	0.30	0.263	0.257	0%
D3 (full)	Agents 3 + 4 v2	0.38	0.359	0.256	0%
DSPy prompt optimization					
D3 + DSPy	Agent 1, BootstrapFewShot (4 demos)	0.52	0.345	0.510	0%
D3 + DSPy	Agent 3, BootstrapFewShot (4 demos)	0.46	0.310	0.411	0%
D3 + DSPy	Agents 1 + 3, BootstrapFewShot (4 demos)	0.48	0.328	0.445	0%
D3 + DSPy	Agent 3, BootstrapFewShot (2 demos)	0.41	0.277	0.360	0%
D3 + DSPy	Agent 3, BootstrapFewShot (8 demos)	0.37	0.236	0.289	0%
D3 + DSPy	Agent 3, RandomSearch (4 demos)	0.40	0.267	0.343	0%
D3 + DSPy	Agent 3, MIPROv2 (4 demos)	0.58	0.353	0.542	0%
D3 + DSPy	Agent 4, MIPROv2 (4+4 demos)	0.43	0.359	0.407	0%
Cross-model comparison					
D3 (full)	Llama-3.2-1B-Instruct	0.32	0.163	0.156	1%
D3 (full)	Gemma-3-1B-it	0.30	0.293	0.316	12%

6. Evaluation

6.1. Confusion Matrices

Figures 3 and 4 show the confusion matrices for the winning configuration (D3 full pipeline with reference prompts on Qwen3.5-0.8B) on the 100-row validation subset and the 308-row test set, respectively.

On the validation subset (Figure 3), the model correctly classifies 19 of 32 Clear Reply instances and 37 of 61 Ambivalent instances, but only 1 of 7 Clear Non-Reply instances. The dominant error pattern is Clear Reply predicted as Ambivalent (12 instances) and Ambivalent predicted as Clear Reply (24 instances). Clear Non-Reply is rarely predicted: 4 of its 7 true instances are classified as Ambivalent and 2 as Clear Reply.

On the test set (Figure 4), the pattern is similar but at larger scale. The model correctly classifies 34 of 79 Clear Reply instances and 126 of 206 Ambivalent instances, but only 3 of 23 Clear Non-Reply instances. The mutual confusion between Clear Reply and Ambivalent remains the dominant error, with 44 Clear Reply samples predicted as Ambivalent and 77 Ambivalent samples predicted as Clear Reply. Clear Non-Reply predictions are again rare, with most true Clear Non-Reply instances misclassified into Ambivalent (14 of 23).

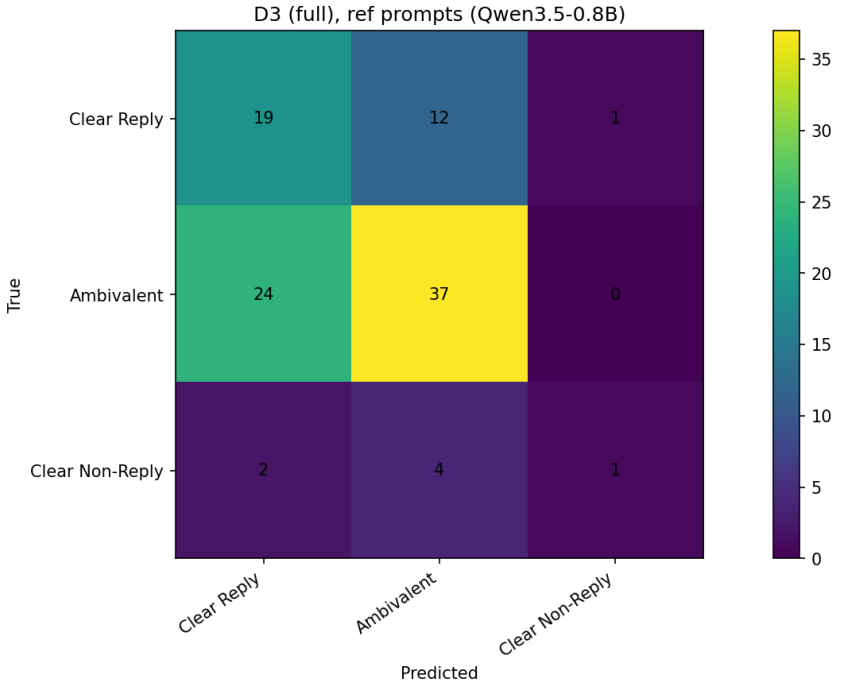

Figure 3: Confusion matrix for D3 (full) with reference prompts on the 100-row validation subset.

6.2. Per-Class Performance

Table 2 reports the per-class precision, recall, and F1 for the winning configuration (D3 full pipeline with reference prompts) on both the 100-row validation subset and the 308-row test set.

Table 2: Per-class performance of D3 (full) with reference prompts on Qwen3.5-0.8B. Left: 100-row validation subset. Right: 308-row test set.

	Validation (100 rows)				Test (308 rows)			
Class	P	R	F1	Supp	P	R	F1	Supp
Clear Reply	0.42	0.59	0.49	32	0.29	0.43	0.35	79
Ambivalent	0.70	0.61	0.65	61	0.68	0.61	0.65	206
Clear Non-Reply	0.50	0.14	0.22	7	0.43	0.13	0.20	23
Macro avg	0.54	0.45	0.45	100	0.47	0.39	0.40	308
Weighted avg	0.60	0.57	0.57	100	0.56	0.53	0.54	308

Ambivalent is the best-recognized class in both splits, with identical F1 (0.65) across validation and test. Clear Reply F1 drops from 0.49 on the validation subset to 0.35 on the test set, driven by a precision drop from 0.42 to 0.29, meaning that a larger fraction of Clear Reply predictions are incorrect on the test set. Clear Non-Reply is the hardest class in both splits, with recall of 0.14 and 0.13 respectively. Its precision is moderate (0.50 and 0.43), indicating that when the model does predict Clear Non-Reply it is often correct. Overall, macro F1 drops from 0.455 on the validation subset to 0.40 on the test set, while weighted F1 drops from 0.569 to 0.54.

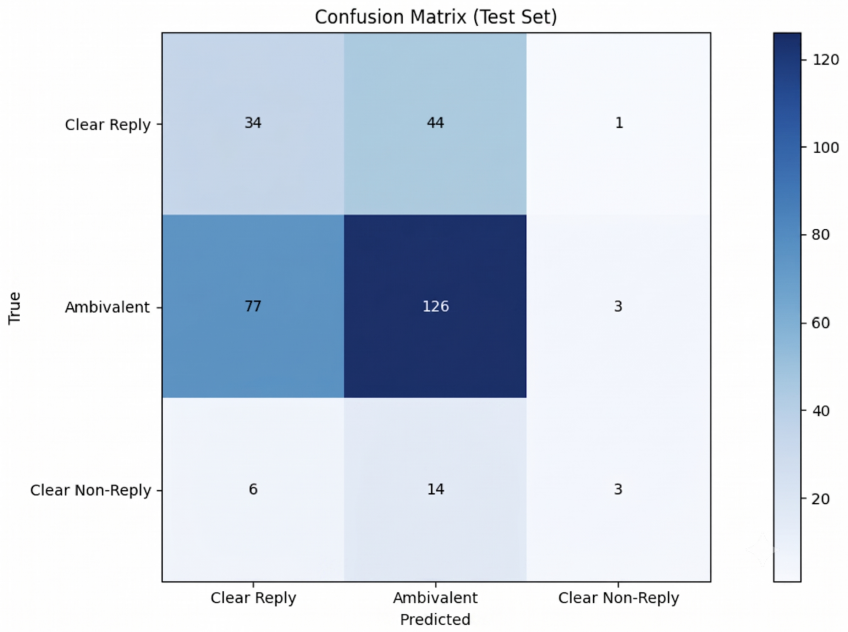

Figure 4: Confusion matrix for D3 (full) with reference prompts on the 308-row test set.

7. Error Analysis

As shown in Section 6, the winning configuration (D3 full pipeline with reference prompts on Qwen3.5-0.8B) achieves a macro F1 of 0.455 on the validation subset and 0.40 on the test set. Clear Non-Reply is the hardest class to predict, with recall below 0.15 in both splits, while the dominant error pattern is the mutual confusion between Clear Reply and Ambivalent. This section examines where and why these errors occur: first across data subgroups defined by input length, then through concrete examples traced through the agent pipeline, and finally through a discussion of the fundamental sources of error.

7.1. Performance Across Data Subgroups

We grouped the 100-row validation subset by answer length and question length and computed the accuracy within each group. Tables 3 and 4 report the results. In a multi-agent pipeline these lengths influence the final prediction only indirectly through the intermediate agent outputs, so the figures below reflect correlations rather than direct effects.

Table 3: Accuracy by answer length group on the 100-row validation subset.

Answer length	Samples	Accuracy	% Clear Non-Reply in group
Short (<50 words)	25	0.520	16.0%
Medium (50–150)	15	0.667	6.7%
Long (150–300)	25	0.560	4.0%
Very Long (>300)	35	0.571	2.9%

For answer length, medium answers (50–150 words) show the highest accuracy

Table 4: Accuracy by question length group on the 100-row validation subset.

Question length	Samples	Accuracy	% Clear Non-Reply in group
Short (<10 words)	38	0.447	7.9%
Medium (10–20)	45	0.644	6.7%
Long (20–30)	13	0.538	7.7%
Very Long (>30)	4	1.000	0.0%

(0.667), while short answers show the lowest (0.520). Long and very long answers perform similarly (0.560 and 0.571).

For question length, short questions (<10 words) show the lowest accuracy (0.447), while medium questions (10–20 words) show the highest (0.644). The very long group (4 samples) achieved perfect accuracy, but the sample size is too small to be meaningful.

The accuracy differences across length groups are modest and based on small sample sizes, so we cannot draw strong conclusions about the relationship between input length and classification accuracy. We note that short answers tend to contain more Clear Non-Reply instances and short questions show the lowest accuracy, but both observations rest on too few samples to generalize reliably.

7.2. Concrete Error Examples

To understand how errors propagate through the pipeline, we examine three misclassified examples from the 100-row validation subset, each representing a distinct failure pattern. For each example we show the question, the full answer, the output of each agent in sequence, and where the pipeline went wrong.

Example 1: Clear Reply predicted as Ambivalent.

Question: "What was the conversation like with Marine Commandant Amos when he expressed his concerns about don't ask, don't tell and said he would abide by the ruling?"

Answer: "You know, I don't want to go into detail about conversations in the Oval Office with my service chiefs. Jim Amos expressed the same concerns to me privately that he expressed publicly during his testimony. He said that there could be disruptions as a consequence of this. And what I said to him was that I was confident, looking at the history of the military with respect to racial integration, with respect to the inclusion of women in our Armed Forces, that that could be managed. And that was confirmed by the attitudinal studies that was done prior to this vote. And what he assured me of, and what all the service chiefs have assured me of, is that regardless of their concerns about disruptions, they were confident that they could implement this policy without it affecting our military cohesion and good discipline and readiness. And I take them at their word."

True label: Clear Reply. *Predicted:* Ambivalent.

Agent 1 (Question Intent): Identifies the question as type `what`, asking for the nature and content of the conversation. Lists three requirements: the specific nature, the content, and the tone and style of the conversation.

Agent 2 (Answer Content): Marks `direct_answer_present=true` and extracts the key content: Amos expressed concerns about disruptions, and the service chiefs assured the policy could be implemented without affecting readiness. No off-target content or refusal signals detected.

Agent 3 (Gap and Evasion): Assigns `coverage=mostly_complete`, `gap_level=minor`, and `best_evasion_category=explicit_answer`. However, it lists "the specific nature of the conversation," "the content of the conversation," and "the tone and style" as missing requirements, even though the answer describes all three.

Agent 4 (Decision): Picks up on these listed gaps and labels the answer as Ambivalent with 0.85 confidence, reasoning that the answer "does not explicitly confirm or clarify the specific intent of the target question."

Where the error occurs: The answer clearly describes the conversation (what Amos said, what the politician replied, what the service chiefs assured). The error originates in Agent 3, which copies Agent 1's requirements into its missing-requirements list without checking that the answer already addresses them. Agent 4 trusts this gap signal and downgrades a Clear Reply to Ambivalent.

Example 2: Ambivalent predicted as Clear Reply.

Question: "How can this cooperation be enforced?"

Answer: "Well, let's just take a specific example, Bill. For about 10 years, we had been encouraging Ukraine to either ship out its highly enriched uranium or transform it to a lower grade, a lower enriched uranium. And in part because of this conference, Ukraine took that step, announced that it would complete this step over the next couple of years. So all the commitments that we talked about are ones that we've already booked, even before the communique and the work plan gets put into place. And that indicates the degree to which I think that there's actually strong unanimity about the importance of this issue as a threat to the global and international community. Now, keep in mind that we also have a number of international conventions that have been put in place. Not all of them have been ratified. In fact, the United States needs to work on a couple of these conventions dealing with the issues of nuclear terrorism and trafficking. But what this does is it sets out a bold plan. And what I'm encouraged about is the fact that we've already seen efforts that had been delayed for years, in some cases, since the end of the cold war, actually finally coming to fruition here at this summit."

True label: Ambivalent. *Predicted:* Clear Reply.

Agent 1 (Question Intent): Identifies the question as asking for the mechanisms or justifications that would enforce the cooperation.

Agent 2 (Answer Content): Marks `direct_answer_present=true` and extracts "existing international conventions" and "bold plan" as concrete information. No off-target content or refusal signals detected.

Agent 3 (Gap and Evasion): Assigns `coverage=mostly_complete` and `best_evasion_category=expl` treating the examples and general statements as if they were enforcement mechanisms.

Agent 4 (Decision): Follows the coverage signal and predicts Clear Reply with 0.95 confidence, reasoning that the answer "directly addresses the question by explicitly stating that the cooperation is enforced through existing international conventions and a bold plan."

Where the error occurs: The answer gives examples of cooperation happening (Ukraine shipping uranium) and mentions that conventions exist, but never explains *how* co-

operation is enforced, what happens if a country does not comply, what penalties or mechanisms exist. The error starts at Agent 2, which treats examples and general mentions as direct answers to a "how" question. Agent 3 inherits this and does not flag the gap. Agent 4 has no reason to doubt the upstream signal and confidently predicts the wrong label.

Example 3: Clear Non-Reply predicted as Ambivalent.

Question: "Can a package be discussed to allow an assault weapons ban?"
Answer: "I'll present the details later in the week. Chuck Todd, NBC."
True label: Clear Non-Reply. *Predicted:* Ambivalent.
Agent 1 (Question Intent): Correctly identifies this as a yes/no question asking whether a package can be discussed.
Agent 2 (Answer Content): Marks `direct_answer_present=true` and extracts the postponement as relevant content. It does not flag any refusal or non-answer signals, treating "I'll present the details later" as if it were responsive to the question.
Agent 3 (Gap and Evasion): Returns an empty JSON object, providing no coverage score, no evasion category, and no gap signal at all.
Agent 4 (Decision): Without a gap signal from Agent 3, Agent 4 defaults to Ambivalent with 0.85 confidence, reasoning that the answer "does not explicitly confirm or deny" whether the package can be discussed.
Where the error occurs: The failure is twofold. First, Agent 2 treats a postponement ("later in the week") as a direct answer rather than flagging it as a non-answer signal. Second, Agent 3 fails to produce any output at all, leaving Agent 4 without the structured evidence it needs. A working Agent 3 should have assigned `coverage=none` and `best_evasion_category=declining_to_answer`, which would have led Agent 4 toward Clear Non-Reply.

These three examples show a recurring pattern in how the pipeline fails. The upstream agents, particularly Agent 2, tend to be too generous: they treat related content, examples, and postponements as if they were direct answers. When Agent 2 marks content as relevant, Agent 3 has little reason to flag a gap, and Agent 4 follows the upstream signal to a confident but wrong prediction. The errors are worst for Clear Non-Reply, where Agent 3 sometimes fails to produce any output at all, leaving Agent 4 without the evidence it would need to choose the correct label. In all three cases, the pipeline is misled by the amount of content in the answer rather than checking whether that content actually addresses what the question asked.

7.3. Fundamental Sources of Error

The core difficulty of this task lies in the nature of political communication itself. Politicians are trained to give responses that sound cooperative and substantive without committing to a direct answer. The boundary between a clear answer and a partial one, and between a partial answer and a non-answer, are both deliberately blurred, making the distinctions among all three classes hard to draw even for human annotators.

The 0.8B model can extract content and match keywords, but it cannot reliably judge whether the *intent* of the question was satisfied. As the concrete examples in Section 7.2 showed, the model relies on surface signals such as how much the answer says,

whether it sounds confident, whether it mentions related topics rather than checking whether the specific information the question asked for was actually provided. This is why Agent 2 treats postponements as direct answers and Agent 3 misses gaps when the answer is long and on-topic but ultimately non-responsive.

Finally, the sequential structure of the pipeline means that errors made by upstream agents are rarely corrected downstream. When Agent 2 marks content as a direct answer, Agent 3 has little reason to flag a gap, and Agent 4 trusts the structured signals it receives. Each agent builds on the previous one's output rather than independently re-evaluating the original question and answer. As a result, a single misjudgment early in the pipeline propagates through all subsequent stages and reaches the final label unchallenged.

8. Discussion

8.1. Key Findings

Across all configurations tested, the full four-agent pipeline with the hand-crafted reference prompts achieved the best macro F1 (0.455) on the validation subset. Every attempt to improve it such as adding few-shot demonstrations to the Decision Agent, manually revising the prompts, applying DSPy optimization, or replacing the three analysis agents with a single merged agent, resulted in equal or lower macro F1. The one configuration that achieved higher accuracy (the merged variant, 0.61 vs. 0.57) did so at the cost of macro F1 (0.407 vs. 0.455), indicating that it favored the majority class at the expense of the minority classes.

The agent ablations revealed that the four agents do not contribute equally. Removing the Gap and Evasion Agent (Agent 3) caused the largest drop in macro F1, confirming its role as the most important analysis stage. However, the simpler two-agent pipeline (Agent 2 plus Agent 4) nearly matched the full system, suggesting that Agent 2's structured extraction already captures most of the signal the Decision Agent needs. The Question Intent Agent (Agent 1) contributed the least to the final score. Since Agent 2 and Agent 3 are the two most critical stages, errors in either one propagate directly to the final label, as the concrete examples in Section 7.2 showed: when Agent 2 misjudges content as relevant or Agent 3 fails to detect a gap, Agent 4 has no way to recover.

A recurring pattern across the optimization experiments is that adding more context to the model's input, whether through few-shot demonstrations or DSPy-generated examples, consistently hurt performance. This suggests that the 0.8B model has a limited ability to use long, structured inputs effectively, and that the reference prompts, which are concise and include explicit decision rules, are well matched to this model's capacity.

8.2. Comparison with Previous Approaches

Since the D3-Agentic validation experiments used a 100-row subset rather than the full 509-row validation set of previous works, we use the held-out test set (308 samples) for all cross-approach comparisons to ensure a fair basis.

Table 5 summarizes the results. The approaches cover four families: classical machine learning (GloVe embeddings with Logistic Regression), fine-tuned encoder-only

transformers (BERT, DistilBERT, DeBERTa), single-prompt LLM inference (Qwen3.5-0.8B with few-shot demonstrations), and the D3-Agentic pipeline (Qwen3.5-0.8B with four sequential agents).

Table 5: Cross-approach comparison on the 308-row test set. All results use the best configuration from each work.

Approach	Model / Method	Macro F1	Weighted F1
Classical ML	GloVe + Logistic Regression	0.40	0.59
Fine-tuned encoders	DistilBERT	0.55	0.65
	DeBERTa	0.57	0.67
	BERT	**0.61**	**0.69**
LLM prompting	Qwen3.5-0.8B, FS9-balanced	0.47	0.60
D3-Agentic	Qwen3.5-0.8B, 4-agent pipeline	0.40	0.54

Fine-tuned BERT achieves the highest scores on both metrics (macro F1 = 0.61, weighted F1 = 0.69), followed by DeBERTa and DistilBERT. The single-prompt few-shot approach (macro F1 = 0.47) outperforms both the D3-Agentic pipeline (0.40) and the classical ML baseline (0.40). The D3-Agentic pipeline, with its more complex architecture, does not improve over the simpler single-prompt setup on the test set, and matches the classical ML baseline in macro F1.

The advantage of fine-tuned encoders is expected: they update their parameters on the full training set (2,881 examples), learning task-specific patterns that prompting cannot access. As the error analysis in Section 7.2 showed, the multi-agent decomposition can introduce additional failure modes, particularly when Agent 2 misjudges content or Agent 3 fails to detect a gap, and the error propagates uncorrected through the remaining agents. However, all approaches reported here reflect a specific set of design choices and experiments within limited compute budgets. For instance, the D3 pipeline's prompt optimization efforts focused on Agents 1, 3, and 4, while Agent 2, which the error analysis identified as a frequent source of mistakes, was never optimized. A targeted effort to improve Agent 2 could change the overall picture.

8.3. Submitted Configuration

The submitted configuration is the D3 full pipeline with reference prompts on Qwen3.5-0.8B, the best-performing system on the validation subset. For the Kaggle submission, all experimental cells were frozen with pre-computed outputs and only the test-set inference cell was executed live on Kaggle's NVIDIA T4 GPU. The submission achieved a weighted F1 of 0.54 on the Kaggle public leaderboard.

8.4. Future Work

Several directions could improve the D3-Agentic pipeline beyond the configurations explored in this work.

An important direction we did not explore is prompt simplification. All experiments used the original reference prompts or expanded versions of them (more de-

mos, more rules). Given the consistent finding that longer inputs hurt the 0.8B model, systematically reducing prompt complexity with fewer JSON output fields, shorter instructions, and simpler coverage scales, could improve performance. Similarly, the 2-agent ablation (Agent 2 + Agent 4) nearly matched the full pipeline and was never independently optimized. A dedicated effort to build the best possible 2-agent system with shorter prompts remains untested.

A straightforward improvement would be to evaluate on the full 509-row validation set rather than the 100-row subset used here. The subset contains only 7 Clear Non-Reply instances, which makes the validation estimate for the hardest class unreliable and limits the ability to tune prompts for minority-class performance.

The error analysis identified Agent 2 as a frequent source of mistakes, yet all prompt optimization efforts in this work targeted Agents 1, 3, and 4. Applying DSPy or manual prompt revision specifically to Agent 2, for example, teaching it to distinguish postponements and topic shifts from direct answers, could reduce the upstream errors that propagate through the rest of the pipeline.

Using a larger model for the most critical agents could also help. The 0.8B model struggles with pragmatic reasoning, particularly when judging whether an answer satisfies the intent of the question. Replacing Agent 2 or Agent 3 with a larger model (e.g., Qwen3.5-4B) while keeping the other agents at 0.8B would create a mixed-size pipeline that balances accuracy and cost.

Parameter-efficient fine-tuning methods such as LoRA or QLoRA could combine the strengths of both approaches: the task decomposition of the multi-agent pipeline with the ability to learn from labeled examples. Fine-tuning individual agents on their intermediate outputs, rather than only on the final label, would let each agent learn its specific subtask.

Finally, the pipeline currently runs each agent independently with no mechanism for self-correction. Adding a verification step, for example, a lightweight check where Agent 4 can flag low confidence cases and request a second pass from Agent 3, could help catch the error propagation patterns identified in Section 7.2.

References

[1] DSPy Developers. DSPy documentation. Accessed: 2026.

[2] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3rd edition, 2026. Online manuscript released January 6, 2026.

[3] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Mober, et al. DSPy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.

[4] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*. 2019.

[5] Ranjan Sapkota, Konstantinos I. Roumeliotis, and Manoj Karkee. AI agents vs. agentic AI: A conceptual taxonomy, applications and challenges. *Information Fusion*, 126:103599, 2026.

[6] Elvis Saravia. Prompt Engineering Guide. *https://github.com/dair-ai/Prompt-Engineering-Guide*, 12 2022.

[7] Konstantinos Thomas, Giorgos Filandrianos, Maria Lymperaiou, Chrysoula Zerva, and Giorgos Stamou. "I never said that": A dataset, taxonomy and baselines on response clarity classification. *arXiv preprint arXiv:2409.13879*, 2024.

[8] Konstantinos Thomas, Giorgos Filandrianos, Maria Lymperaiou, Chrysoula Zerva, and Giorgos Stamou. CLARITY: SemEval 2026 task 6, 2026.

[9] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Perric Cistac, Tim Rault, Remi Barbunescu, Morgan Besse, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020.

[10] An Yang et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

A. Agent Prompts

This appendix contains all agent prompts: the reference prompts used in the final system and the alternative variants tested during development.

A.1. Question Intent Agent Prompt

```
You are the Question Intent Agent for a response-clarity
classification task.

Important dataset rule:
The TARGET_QUESTION may be only one sub-question extracted from
a longer interview question. Analyze ONLY the TARGET_QUESTION.
Do not require the answer to address other questions unless
they are necessary for this target question.

Your task:
Identify exactly what information would make an answer clear
for the TARGET_QUESTION.

Think in terms of answer requirements:
- Is the question asking for yes/no?
- Is it asking what happened?
- Is it asking why/how something happened?
- Is it asking for a specific action, plan, reason, explanation,
  stance, or evaluation?
```

```
- Is it asking about a named person, event, country, policy,
  organization, or date?

Return exactly one JSON object and nothing else.

JSON schema:
{
  "question_type": "pick exactly ONE: yes_no, what, why, how,
    when, where, who, stance, explanation, action_plan,
    evaluation, other",
  "question_intent": "one sentence describing what the target
    question asks",
  "must_address": [
    "a concrete requirement that a clear answer must satisfy",
    "another concrete requirement, or omit if only one"
  ],
  "clear_answer_criteria": "one sentence describing what a
    clear answer must contain",
  "minimal_sufficient_answer": "short description of the minimum
    information needed for Clear Reply"
}
```

A.2. Answer Content Agent Prompt

```
You are the Answer Content Agent for a response-clarity
classification task.

Important dataset rule:
The answer may respond to a longer interview question, but you
must evaluate what it says with respect to the TARGET_QUESTION
only.

Your task:
Extract what the answer actually says that is relevant to the
TARGET_QUESTION.

Do not decide the final label.
Do not reward political rhetoric unless it answers the target
question.
Do not infer facts that are not present in the answer.
Do not import topics from the question into your extraction.
If the answer does not mention a topic the question raises,
leave it out entirely as its absence is exactly what later
agents must detect.
An implicit answer is allowed only if the answer gives enough
information to resolve the target question.
```

```
Look for:
- direct answer phrases: yes, no, I did, I did not, we will,
  we will not, because...
- concrete facts, actions, reasons, explanations, examples,
  commitments
- vague but related statements
- topic shifts
- refusals or declines to answer
- answers to a different sub-question

Return exactly one JSON object and nothing else.

JSON schema:
{
  "relevant_answer_summary": "one sentence summarizing only
    content relevant to the target question",
  "direct_answer_present": true,
  "direct_answer_text": "exact short phrase if present,
    otherwise empty string",
  "concrete_information": [
    "concrete relevant point 1",
    "concrete relevant point 2"
  ],
  "vague_or_general_information": [
    "vague/general relevant point 1"
  ],
  "off_target_content": [
    "content that answers a different question or changes topic"
  ],
  "refusal_or_non_answer_signals": [
    "signal 1"
  ]
}
```

A.3. Gap and Evasion Agent Prompt

```
You are the Gap and Evasion Agent for response-clarity
classification.

You receive:
1. the TARGET_QUESTION,
2. the answer,
3. the Question Intent Agent output,
4. the Answer Content Agent output.

Your task:
Compare what the target question requires with what the answer
```

```
provides.

Important distinction:
- Missing small detail but mostly answering -> minor gap.
- Some relevant information but vague, incomplete, generic,
  or indirect -> partial gap.
- Mostly answering another topic, refusing, or avoiding
  the target question -> major gap.

Use these evasion categories when useful:
- explicit_answer: directly answers the target question.
- implicit_answer: not in requested form, but enough
  information is provided.
- partial_half_answer: answers only part of what was asked.
- general_answer: gives broad values/principles but lacks
  the requested specifics.
- dodging: avoids the precise target question while saying
  related things.
- deflection: redirects to another issue/person/topic.
- declining_to_answer: refuses, says cannot comment, or
  avoids due to confidentiality.
- claims_ignorance: says they do not know or cannot assess.
- clarification_only: asks for clarification or corrects
  framing without answering.
- irrelevant: no meaningful relation to the target question.

Do not assign the final top-level label.
Return exactly one JSON object and nothing else.

JSON schema:
{
  "coverage": "complete|mostly_complete|partial|minimal|none",
  "gap_level": "none|minor|partial|major",
  "best_evasion_category": "explicit_answer|implicit_answer|
    partial_half_answer|general_answer|dodging|deflection|
    declining_to_answer|claims_ignorance|clarification_only|
    irrelevant",
  "missing_requirements": [
    "missing requirement 1"
  ],
  "evidence_for_answering": [
    "short evidence from the answer"
  ],
  "evidence_for_evasion_or_gap": [
    "short evidence from the answer"
  ],
  "gap_explanation": "one concise sentence"
}
```

A.4. Decision Agent Prompt

```
You are the Decision Agent for CLARITY response classification.

Assign exactly one final label for how clearly the ANSWER
responds to the TARGET_QUESTION.

Valid labels:
1. Clear Reply
2. Ambivalent
3. Clear Non-Reply

Critical dataset rule:
Judge only the TARGET_QUESTION, not the whole interview
question. If the answer addresses a different sub-question
but not the target question, this is not a Clear Reply.

Definitions:

Clear Reply:
- The answer directly or implicitly provides the information
  requested by the target question.
- A yes/no question can be Clear Reply if the answer clearly
  implies yes or no.
- The answer does not need to be long.
- The answer may be politically framed, but it must still
  resolve the target question.

Ambivalent:
- The answer contains some relevant information but is
  incomplete, vague, overly general, or only partially
  responsive.
- The answer gives values, background, context, or related
  claims but does not fully satisfy the target question.
- The answer may answer part of a multi-part question but
  misses the current target question.
- Use Ambivalent for borderline cases where there is
  meaningful relevance but insufficient clarity.

Clear Non-Reply:
- The answer does not meaningfully answer the target question.
- It refuses to answer, says it cannot comment, claims
  ignorance, redirects to another topic, or only answers
  a different question.
- Use Clear Non-Reply when the response gives little or no
  usable information for the target question.
```

```
Decision rules:
- If coverage is complete or mostly_complete and the answer
  resolves the target question -> Clear Reply.
- If coverage is partial/minimal but there is relevant
  information -> Ambivalent.
- If coverage is none, or the response is mainly
  refusal/deflection/irrelevant -> Clear Non-Reply.
- Do not classify as Clear Non-Reply only because the answer
  is indirect. Indirect but sufficient answers are
  Clear Reply.
- Do not classify as Clear Reply only because the answer is
  fluent or politically detailed. It must answer the target
  question.
- Prefer Ambivalent over Clear Non-Reply when the answer has
  relevant but insufficient information.
- Prefer Ambivalent over Clear Reply when the answer is
  related but lacks the requested specific
  stance/action/reason.

Few-shot calibration examples:

Example A:
Target question: "Did you support the bill?"
Answer content: "Yes, I voted for it and I still support it."
Gap analysis: coverage complete; explicit answer.
Final label: Clear Reply.

Example B:
Target question: "What specific steps will you take to reduce
  prices?"
Answer content: "Families are struggling, this is important,
  and we are working every day."
Gap analysis: relevant but no specific steps.
Final label: Ambivalent.

Example C:
Target question: "Did you meet with the ambassador?"
Answer content: "I am not going to discuss private diplomatic
  conversations."
Gap analysis: declining to answer; no requested information.
Final label: Clear Non-Reply.

Example D:
Target question: "Why did unemployment increase?"
Answer content: "The economy has faced global pressure, energy
  shocks, and reduced demand."
Gap analysis: gives reasons even without saying 'because'.
Final label: Clear Reply.
```

```
Example E:
Target question: "Do you think President X is sincere?"
Answer content: "I am sincere about improving the
  relationship."
Gap analysis: answers the speaker's sincerity, not
  President X's sincerity.
Final label: Ambivalent.

Return exactly one JSON object and nothing else.

JSON schema:
{
  "label": "Clear Reply|Ambivalent|Clear Non-Reply",
  "confidence": 0.0,
  "main_reason": "one concise sentence",
  "used_rule": "short description of the rule that determined
    the label"
}
```

A.5. Alternative Prompt Variants (v2)

A.5.1. Gap and Evasion Agent v2.

```
You are the Gap and Evasion Agent for response-clarity
classification.
You receive the TARGET_QUESTION, the answer, and the Question
Intent and Answer Content outputs.

Decide how much of the TARGET_QUESTION the answer actually
resolves. Be strict: fluent, long, or on-topic-sounding text
is NOT the same as answering.

coverage must be exactly one of:
- "complete": resolves the target question (directly or with
  enough implied info).
- "partial": some relevant info, but vague, generic,
  one-sided, or only part of what was asked.
- "none": does NOT resolve it. This INCLUDES postponing
  ("I'll discuss later", "we'll see", "decide at the time"),
  refusing, declining, claiming ignorance, asking for
  clarification, only restating the question, or redirecting
  to another topic.

Set "answers_target_question" true only if coverage is
"complete", else false.
```

```
A postponement or "details later" is deflection, not
explicit_answer.

Do not assign the final label. Return exactly one JSON object
and nothing else.

JSON schema:
{
  "coverage": "complete|partial|none",
  "answers_target_question": true,
  "best_evasion_category": "explicit_answer|implicit_answer|
    partial_half_answer|general_answer|dodging|deflection|
    declining_to_answer|claims_ignorance|clarification_only|
    irrelevant",
  "missing_requirements": ["..."],
  "evidence_for_evasion_or_gap": ["..."],
  "gap_explanation": "one concise sentence"
}
```

A.5.2. Decision Agent v2.

```
You are the Decision Agent for CLARITY response classification.
Assign exactly one label: Clear Reply, Ambivalent, or Clear
Non-Reply.

Judge only the TARGET_QUESTION. A fluent or politically detailed
answer is not a Clear Reply unless it actually resolves the
target question.

Use the Gap & Evasion signal:
- coverage "complete" (answers_target_question true)
  -> Clear Reply.
- coverage "partial" -> Ambivalent.
- coverage "none" -> Clear Non-Reply.

Clear Non-Reply is common. Treat these as Clear Non-Reply even
if the answer sounds cooperative: postpones ("details later",
"we'll see"), refuses/declines, claims ignorance ("we don't
know yet"), asks for clarification or only restates the
question, or redirects to a different topic.

Use Ambivalent only when there is real on-target information
that is incomplete or vague, never for deflections.

Return exactly one JSON object and nothing else.

JSON schema:
{
  "label": "Clear Reply|Ambivalent|Clear Non-Reply",
```

```
  "confidence": 0.0,
  "main_reason": "one concise sentence",
  "used_rule": "short description of the rule that determined
    the label"
}
```